

Basic Algorithms

Fibonacci Number

Input: n and **Output:** n^{th}

Fibonacci number.

Algorithm: assume a as
first(previous) and b as
second(current) numbers

```
fib(n){  
    a = 0, b= 1, f=1 ;  
    for(i = 2 ; i <=n ; i++)  
    {  
        f = a+b ;  
        a=b ;  
        b=f ;  
    }  
    return f ;  
}
```

Time Complexity:

The algorithm above iterates up to $n-1$ times, so time complexity is $O(n)$.

Space Complexity

4 variables
 $S(n)=O(1)$

Basic Algorithms : GCD

Algorithm: assume (for simplicity) $a > b \geq 0$

gcd(a,b)

{

While(b != 0)

{

d = a/b ;

temp = b ;

b = a - b * d ;

a = temp ;

}

return a ;

}

Time Complexity:

If the given numbers a and b are of n-bits then while loop executes for n time and the division and multiplication can be done in $O(n^2)$ time. So the total running time becomes $O(n^3)$.

Space Complexity: The only allocated spaces are for variables so space complexity is **$O(1)$** .

Another way :

For Simplicity Let us assume that

$$b = 2^n$$

$$n = \log b$$

Loop executes $\log b$ times in worst case

Time Complexity = $O(\log b)$

Searching Algorithm :

Sequential Search and its analysis

LinearSearch(A, n, key)

```
{  
    for(i=0;i<n ; i++)  
    {  
        if(A[i] == key)  
        {  
            return i;  
        }  
    }  
    return -1;//-1 indicates unsuccessful search  
}
```

Time complexity = $O(n)$

Space Complexity = initial
input + some variables = $O(n)$

Sorting Algorithms

Internal Sorting
algorithms, which
assume that data is
stored in an array in
main memory,

**External Sorting
Algorithm,** which assume
that data is stored on disk
or some other device that
is best accessed
sequentially

In-place: The algorithm
uses no additional array
storage, and hence (other
than perhaps the system's
recursion stack) it is
possible to sort very large
lists without the need to
allocate additional
working storage

Stable: A sorting algorithm is stable if two elements that are
equal remain in the same relative position after sorting is
completed. This is of interest, since in some sorting applications
you sort first on one key and then on another. It is nice to know
that two items that are equal on the second key, remain sorted
on the first key.

Sorting Algorithms

Internal Sorting
algorithms, which
assume that data is
stored in an array in
main memory,

**External Sorting
Algorithm,** which assume
that data is stored on disk
or some other device that
is best accessed
sequentially

In-place: The algorithm
uses no additional array
storage, and hence (other
than perhaps the system's
recursion stack) it is
possible to sort very large
lists without the need to
allocate additional
working storage

Stable: A sorting algorithm is stable if two elements that are
equal remain in the same relative position after sorting is
completed. This is of interest, since in some sorting applications
you sort first on one key and then on another. It is nice to know
that two items that are equal on the second key, remain sorted
on the first key.

Sorting Algorithms : Bubble Sort

BubbleSort(A, n)

```
{
    for(i = 0; i < n-1; i++)
    {
        for(j = 0; j < n-i-1; j++)
        {
            if(A[j] > A[j+1])
            {
                temp = A[j];
                A[j] = A[j+1];
                A[j+1] = temp;
            }
        }
    }
}
```

Time Complexity:

Inner loop executes for (n-1) times when i=0, (n-2) times when i=1 and so on:

Time complexity

$$= (n-1) + (n-2) + (n-3) + \dots + 2 + 1$$

$$= O(n^2)$$

There is no best-case linear time complexity for this algorithm.

Space Complexity:

Since no extra space besides 3 variables is needed for sorting

$$\text{Space complexity} = O(n)$$

Sorting Algorithm : Bubble Sort

Tracing: 3, 6, 1, 8, 4, 5

BubbleSort(A, n)

```
{
    for(i = 0; i < n-1; i++)
    {
        for(j = 0; j < n-i-1; j++)
        {
            if(A[j] > A[j+1])
            {
                temp = A[j];
                A[j] = A[j+1];
                A[j+1] = temp;
            }
        }
    }
}
```

i=0,

j=0, is A[j]>A[j+1] ? : no

J=1, is A[j]>A[j+1] ? : Yes

Swap (A[j],A[j+1])

J=2, is A[j]>A[j+1] ? : no

J=3, is A[j]>A[j+1] ? : yes

Swap(A[j],A[j+1])

J=4, is A[j]>A[j+1] ? : yes

Swap(A[j],A[j+1])

8 in its final pos,

J=5,j loop exit, goes for i=1



Sorting Algorithm : Selection Sort

Time Complexity:

SelectionSort(A)

```
{
for( i = 0; i < n ; i++)
{
    min_idx=i;
    {
    for ( j = i + 1; j < n ; j++)
        if (A[j] < A[min_idx])
            min_idx=j;

    }

    swap(A[i],A[min_idx]);
}
}
```

Inner loop executes for (n-1) times when i=0, (n-2) times when i=1 and so on:

$$\begin{aligned} \text{Time complexity} &= (n-1) + (n-2) + (n-3) + \\ &\dots\dots\dots + 2 + 1 \\ &= O(n^2) \end{aligned}$$

There is no best-case linear time complexity for this algorithm, but number of swap operations is reduced greatly.

Space Complexity:

Since no extra space besides 5 variables is needed for sorting

$$\text{Space complexity} = O(n)$$

Sorting Algorithm : Selection Sort

SelectionSort(A)

```
{
for( i = 0; i < n ; i++)
{
    min_idx=i;
    {
        for ( j = i + 1; j < n ; j++)
            if (A[j] < A[min_idx])
                min_idx=j;
    }

    swap(A[i],A[min_idx]);
}
}
```

Tracing:

3	6	1	8	4	5
---	---	---	---	---	---

i=0

Min_idx=0

J=1

Is a[j]<a[min_idx]: no

3	6	1	8	4	5
---	---	---	---	---	---

J=2, is a[j]<a[min_idx]: yes

3	6	1	8	4	5
---	---	---	---	---	---

Min_idx=j

J=3, is a[j]<a[min_idx]: no

3	6	1	8	4	5
---	---	---	---	---	---

J=4, is a[j]<a[min_idx]: no

3	6	1	8	4	5
---	---	---	---	---	---

J=5, is a[j]<a[min_idx]: no

3	6	1	8	4	5
---	---	---	---	---	---

J=6, j loop exit

3	6	1	8	4	5
---	---	---	---	---	---

Swap(a[j], a[min_idx])

1	6	3	8	4	5
---	---	---	---	---	---

Sorting Algorithm : Insertion Sort

InsertionSort(A)

```
{
    for (i=1;i<n;i++)
    {
        key = arr[ i];
        for(j=i; j>0 && arr[j-1] >key; j--)
        {
            arr[j ] = arr[j-1];
        }
        arr[j ] = key;
    }
}
```

Time Complexity:

Worst Case Analysis:

Array elements are in reverse sorted order

Inner loop executes for 1 times when $i=1$, 2 times when $i=2...$ and $n-1$ times when $i=n-1$:

$$\begin{aligned}\text{Time complexity} &= 1 + 2 + 3 + \dots + (n-2) + (n-1) \\ &= O(n^2)\end{aligned}$$

Best case Analysis:

Array elements are already sorted

Inner loop executes for 1 times when $i=1$, 1 times when $i=2...$ and 1 times when $i=n-1$:

$$\begin{aligned}\text{Time complexity} &= 1 + 1 + 1 + \dots + 1 + 1 \\ &= O(n)\end{aligned}$$

Space Complexity:

Since no extra space besides 5 variables is needed for sorting

$$\text{Space complexity} = O(n)$$

Sorting Algorithm : Insertion Sort

Trace:

InsertionSort(A)

{

for (i=1;i<n;i++)

{

key = arr[i];

for(j=i; j>0 && arr[j-1] >key; j--)

{

arr[j] = arr[j-1];

}

arr[j] = key;

}

}

i=1

Key=a[i]=6

j=i=1 , is j>0,a[j-1]>key no, j loop exit

a[j]= key= a[i]=6

i=2,

Key=a[i]=1,

j=2 , is a[j-1]>key: yes

a[j]=a[j-1]=6

J=1, is a[j-1]>key: yes

a[j]=a[j-1]=3

J=0, j loop exits

a[0]=key = 1

One in its final position

3	6	1	8	4	5
---	---	---	---	---	---

3	6	1	8	4	5
---	---	---	---	---	---

3	6	1	8	4	5
---	---	---	---	---	---

3	6	6	8	4	5
---	---	---	---	---	---

3	6	6	8	4	5
---	---	---	---	---	---

3	3	6	8	4	5
---	---	---	---	---	---

1	3	6	8	4	5
---	---	---	---	---	---